

Cahier des Charges : Application de Système de Transfert D'argent

1. Présentation Générale

Cette application permet aux utilisateurs de :

- Créer un compte sécurisé avec vérification OTP.
- Se connecter à leur compte.
- Recharger leur solde via une agence.
- Envoyer de l'argent à un autre utilisateur de l'application.

L'application sera développée en équipe via Git.

2. Objectifs

- Offrir une interface simple et sécurisée pour la gestion de Transfert D'argent.
- Assurer une vérification des comptes via OTP (One-Time Password).
- Intégrer des fonctionnalités essentielles : création de compte, authentification, recharge, transfert d'argent.
- Déployer une solution contenue dans Docker pour faciliter le développement et le déploiement.

3. Fonctionnalités

A. Création de Compte

1. L'utilisateur renseigne son nom, prénom et numéro de téléphone.
2. Le système envoie un code OTP au numéro renseigné.
3. L'utilisateur saisit le code OTP pour vérification.
4. Si le code est correct, il choisit un mot de passe.
5. Le compte est créé.

B. connexion

1. L'utilisateur saisit son numéro et son mot de passe pour accéder à son compte.

C. rechargement de compte

1. L'utilisateur se rend dans une agence physique.
2. Il demande à recharger son compte d'un certain montant.

D. envoie d'argent

1. L'utilisateur sélectionne un destinataire.
2. Il saisit le montant à envoyer et clique sur "Envoyer".
3. Une vérification du solde est effectuée avant validation.

4. Architecture Technique

Fontend : angular

- Interface utilisateur responsive et interactive.
- Connexion aux APIs backend via HTTP.
- Gestion des formulaires de création, connexion et transactions.

Backend : Django (python)

- API REST pour gérer l'ensemble des opérations.
- Sécurité via tokens (JWT).
- Gestion de la base de données et des règles métiers.

Base de données : MySQL

- Stockage des utilisateurs, OTP, soldes, transactions.
- Relations sécurisées entre les entités.

Conteneurisation : Docker

- Un conteneur pour Angular, un autre pour Django, un pour MySQL.
- Docker Compose pour orchestrer les services.

Dépôt Git

- Hébergement du code source (GitHub)
- Gestion collaborative avec branches.

5. Modèle de Données Simplifié

- **Utilisateur** : id, nom, prénom, numéro, password (hash), solde.
- **OTP** : id, code, numéro, expiration.

- **Transaction** : id,envoi_id, receveur_id, montant, date.

6. Sécurité

- Hachage des mots de passe.
- Expiration des OTP. • Vérification de l'identité avant chaque opération.
- Protection contre les injections SQL.

7. Travail Collaboratif

- Utilisation de Git avec des branches de fonctionnalités.
- Revue de code avant fusion dans la branche principale.
- Utilisation de tâches pour la répartition du travail (GitHub).